System Calls for Directory Management

Definition

Directory-management system calls are system calls used to create, remove, link, mount, and unmount directories or file systems. These calls deal with directories and the file system as a whole, rather than with only one file.

1. Basic Idea

Main Concept

Some system calls are mainly concerned with:

- directories,
- file-system structure,
- merging file systems,
- and shared file naming.

These are different from file-management calls that work on one specific file.

2. `mkdir()` and `rmdir()`

Creating and Removing Directories

The first two important directory-management calls are:

- `mkdir()` — creates a new directory
- `rmdir()` — removes an empty directory

Note

`rmdir()` removes only an **empty** directory.

3. `link()` System Call

Meaning of `link`

The `link()` system call allows the **same file** to appear under two or more names, possibly in different directories.

Purpose

A common use of `link()` is file sharing. For example, several members of a programming team may all use the same file from their own directories, even under different names.

Shared File vs Copy

A **shared file** is not the same as making private copies:

- In a shared file, there is only **one actual file**.
- Any change made by one user becomes visible immediately to others.
- In separate copies, later changes in one copy do not affect the others.

3.1 Example of `link()`

Example

```
link("/usr/jim/memo", "/usr/ast/note");
```

This creates a new directory entry so that:

$$/usr/jim/memo \quad \text{and} \quad /usr/ast/note$$

refer to the same file.

4. How `link()` Works

i-number

Every file in UNIX has a unique number called an **i-number**. This identifies the file and points to its i-node entry.

Directory Structure

Conceptually, a directory is a collection of:

$$(\text{i-number}, \text{ASCII name})$$

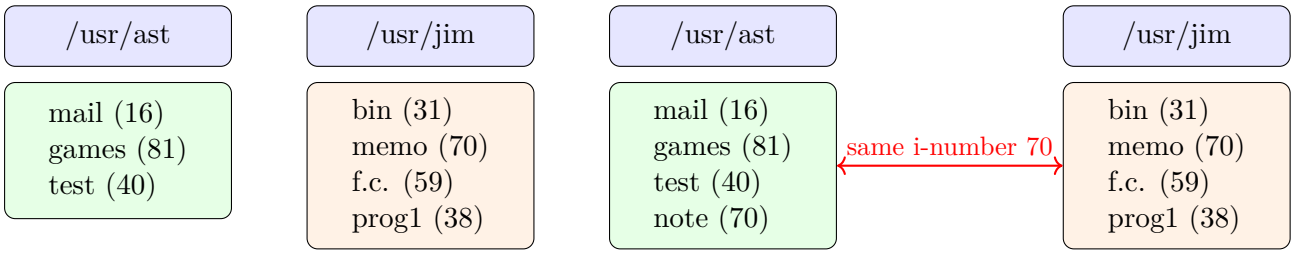
pairs.

What `link()` Actually Does

The `link()` call creates a **new directory entry** with a new name, but it uses the **same i-number** as an already existing file.

4.1 Before and After Linking

(a) Before Linking **(b) After Linking**



Result

After linking, two directory entries have the same i-number, so they refer to the same file.

5. `unlink()` System Call

Meaning of `unlink`

If one of the names of a linked file is removed using `unlink()`, the other name still remains.

When File is Really Deleted

UNIX removes the actual file from disk only when:

- all directory entries pointing to it are removed, and
- no links remain.

6. `mount()` System Call

Meaning of `mount`

The `mount()` system call allows two separate file systems to be merged into one single directory tree.

Typical Use

A common example is:

- main system files on one hard-disk partition,
- user files on another partition,
- USB drive inserted later and attached into the same tree.

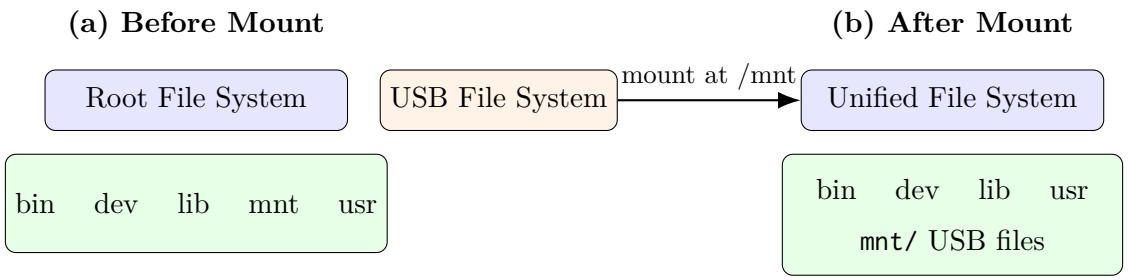
Example

```
mount("/dev/sdb0", "/mnt", 0);
```

Parameters of `mount`

- first parameter — block special file of the device
- second parameter — mount point in the directory tree
- third parameter — whether mounted read-write or read-only

6.1 Before and After Mounting



Main Advantage

After mounting, files on the new device can be accessed by normal path names from the root directory or working directory. The user does not need to worry about which physical device contains the file.

7. Integrated File Hierarchy

Single Tree

The `mount()` call makes it possible to integrate:

- removable media,
- hard-disk partitions,
- external hard disks,
- USB sticks

into one single file hierarchy.

Benefit

This gives a clean and device-independent way to access files.

8. `umount()` System Call

Meaning of `umount`

When a mounted file system is no longer needed, it can be detached from the file tree using the `umount()` system call.

Purpose

`umount()` safely removes the attached file system from the integrated hierarchy.

9. Main Calls at a Glance

`Call`	Purpose
`mkdir()`	Create a new directory
`rmdir()`	Remove an empty directory
`link()`	Create another name for the same file
`unlink()`	Remove a file name (directory entry)
`mount()`	Attach one file system to another
`umount()`	Detach a mounted file system

10. Conclusion

Summary

Directory-management system calls control the structure of directories and the file system as a whole. They allow directories to be created or removed, files to be shared through links, and multiple file systems to be combined into one integrated hierarchy through mounting.